基于FLASK框架的AI低误报智能火灾预警系统

核心设计文档

目录

一、系统概述 3

1.1 系统设计意义 3

1.2 当前火灾预警系统的现状 3

二、系统开发关键技术 4

2.1 FLASK框架 4

2.2 多传感器数据采集技术 4

2.3 AI智能算法技术 5

2.4 网络通信技术 5

2.5 数据库技术 5

三、系统核心需求 6

3.1 数据采集与预处理需求 6

3.2 AI智能识别与预警需求 6

3.3 实时监控与远程控制需求 6

四、 系统设计 7

4.1 系统总体架构设计 7

4.1.1 感知层 7

4.1.2 网络层 7

4.1.3 平台层 7

4.1.4 应用层 7

4.2 系统模块设计 7

4.2.1 数据采集与预处理模块 7

4.2.2 AI 智能识别与预警模块 8
4.2.3 实时监控与远程控制模块 8
4.3 数据库设计 8
4.3.1 数据库设计概述 8
4.3.2 主要的数据表结构 8
五、 使用说明 11
5.1 使用说明 11
5.1.1 主机统计 11
5.1.2 设备历史数据 14
5.1.3 从机数据监控 15
5.1.4 AI 数据分析建议页面 17

一、系统概述

1.1 系统设计意义

在工业生产、商业建筑及居民住宅等各类场景中，火灾事故的发生往往会带来严重的人员伤亡与财产损失。传统火灾预警系统多依赖单一传感器检测，易受环境干扰，存在误报率高、预警滞后等问题，不仅降低了预警系统的可信度，还可能导致相关人员对预警信息产生麻痹心理，延误火灾初期的扑救时机。

随着人工智能技术与物联网的快速发展，融合多维度数据感知与智能分析的预警系统成为解决传统问题的关键。本系统基于 FLASK 框架开发，整合多类型传感器数据采集、AI 智能算法分析、实时通信与远程监控功能，构建 AI 低误报智能火灾预警系统。该系统能够通过多传感器数据融合与 AI 模型精准识别火灾特征，大幅降低误报率；同时实现火灾隐患的实时监测、智能预警与远程管控，为火灾预防与应急处置提供高效支持，保障人员生命与财产安全，具有重要的实际应用价值与社会意义。

1.2 当前火灾预警系统的现状

近年来，火灾预警系统市场需求持续增长，各类场景对预警系统的精准性、实时性与智能化要求不断提升。目前市场上的火灾预警产品主要分为传统型与智能型两类：传统型系统多采用单一传感器，通过阈值判断触发报警，结构简单但误报率高，难以适应复杂环境；智能型系统开始融合多传感器数据与初步算法分析，但部分产品存在数据处理效率低、AI 模型泛化能力不足、预警响应滞后等问题。

从应用场景来看，工业场景对预警系统的抗干扰能力与精准定位要求较高，而居民住宅与商业建筑更注重系统的便捷性与低误报率。当前主流的智能预警系统多基于物联网技术实现数据传输，但在 AI 算法优化、多维度数据融合分析、跨平台交互等方面仍有提升空间。随着 5G、边缘计算与 AI 深度学习技术的发展，具备低误报、高实时性、智能化决策的火灾预警系统成为行业发展趋势，市场应用前景广阔。

二、系统开发关键技术

本系统采用“感知层-网络层-平台层-应用层”的四层架构设计，后端核心框架为 FLASK，结合多传感器数据采集、AI 算法分析、实时通信等关键技术，保障系统的稳定性、精准性与易用性。如图 1 所示。

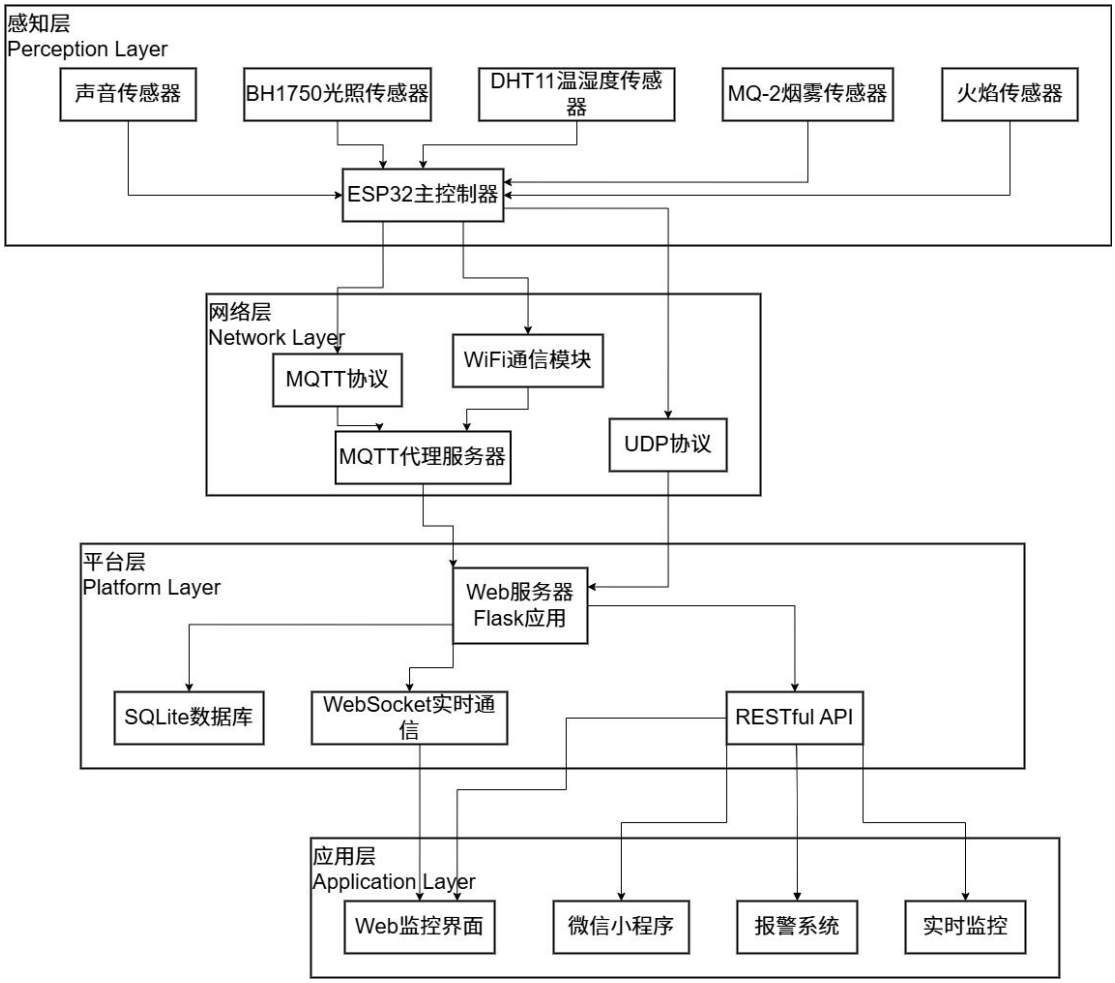

图 1 系统架构设计图

2.1 FLASK 框架

FLASK 作为一款基于 Werkzeug WSGI 工具包与 Jinja2 模板引擎的轻量级 Python Web 框架，具备灵活、简洁、易扩展的核心特点，在本系统中承担着后端核心的重要职责，不仅能提供 RESTful API 接口以实现前端与后端、感知层与平台层的数据交互，还能整合 SQLite 数据库完成传感器数据、预警记录、用户信息等数据的存储与管理，同时支持 WebSocket 实时通信协议保障传感器数据与预警信息的实时推送，更便于集成 AI 算法模型与微信小程序接口、报警服务等第三方服务，有效降低系统开发与维护成本。

2.2 多传感器数据采集技术

多传感器数据采集技术是系统获取火灾相关数据的基础，系统感知层整合了多种高精度传感器以实现多维度数据的全面采集，为 AI 分析提供丰富数据源，其中火焰传感器可检测环境中的火焰辐射信号并精准识别明火特征，MQ-2 烟雾传感器能监测环境中烟雾浓度并区分正常烟雾与火灾烟雾的浓度阈值差异，DHT11 温湿度传感器可实时采集环境温度与湿度数据以排除因温湿度异常波动导致的误报，BH1750 光照传感器能辅助判断环境光照变化并结合火焰传感器数据提升明火识别准确率，声音传感器则可采集环境中的爆炸声、燃烧噼啪声等异常声音，进一步丰富火灾特征数据，所有传感器均通过 ESP32 主控制器进行数据汇总与初步处理，确保数据采集的稳定性与同步性。

2.3 AI 智能算法技术

AI 智能算法技术是实现系统低误报的核心，系统采用多数据融合与深度学习模型对传感器采集的多维数据进行智能分析，流程涵盖数据预处理、特征提取、深度学习模型构建及实时推理等关键环节，其中数据预处理阶段会对传感器数据进行降噪、归一化处理并剔除异常数据点以保障数据质量，特征提取阶段从温度、湿度、烟雾浓度、光照强度、声音信号等多维数据中提取温度骤升速率、烟雾浓度持续上升趋势、火焰辐射强度等火灾特征因子，深度学习模型基于 CNN（卷积神经网络）与 LSTM（长短期记忆网络）融合模型，通过历史火灾数据与正常环境数据的训练构建火灾识别模型，能够精准区分火灾信号与环境干扰信号，大幅降低误报率，实时推理阶段则将预处理后的实时传感器数据输入训练好的 AI 模型，快速完成火灾风险判断与预警等级评估。

2.4 网络通信技术

网络通信技术为系统各层级间的数据传输提供保障，系统网络层采用多种通信协议协同工作以确保数据传输的实时性与可靠性，其中 MQTT 协议用于实现传感器与 MQTT 代理服务器之间的低功耗、高可靠数据传输，适用于物联网设备的远程通信场景，WiFi 通信模块支持 ESP32 主控制器与网络层的无线连接，可灵活适配不同场景的部署需求，UDP 协议用于部分实时性要求高的数据传输以提升数据传输效率，WebSocket 协议则实现平台层与应用层的实时双向通信，保障预警信息、监控数据的实时推送。

2.5 数据库技术

数据库技术承担系统数据存储与管理的核心职责，系统采用具有轻量级、无需单独部署、支持多平台、操作简单等特点的 SQLite 数据库作为数据存储核心，适用于本系统的中小型数据存储需求，具体存储内容包括传感器实时采集的数据、火灾预警记录、设备状态信息等，同时 SQLite 支持数据的快速查询、插入、更新与删除操作，有效保障系统数据处理效率。

三、系统核心需求

3.1 数据采集与预处理需求

数据采集与预处理需求主要包括以下核心内容，一是支持火焰、烟雾、温湿度、光照、声音等多传感器的数据实时采集，且采集频率可灵活配置，默认采集频率为 1 次/秒；二是具备完善的数据预处理功能，通过数据降噪、异常值剔除、数据同步等操作保障采集数据的准确性，为后续 AI 分析提供高质量数据源。

3.2 AI 智能识别与预警需求

AI 智能识别与预警需求聚焦于火灾识别的精准性与预警的及时性，首先需基于 AI 模型实现火灾信号的精准识别，确保误报率低于 3%；其次要支持预警等级的清晰划分，具体分为一级预警（疑似火灾）、二级预警（确认火灾），不同预警等级需对应不同的报警方式，实现差异化预警；同时具备预警信息快速推送功能，支持 Web 界面弹窗、微信小程序消息等多种推送渠道，确保相关人员能及时接收预警信息。

3.3 实时监控与远程控制需求

实时监控与远程控制需求旨在实现对火灾相关情况的远程掌控与快速响应，支持通过 Web 监控界面与微信小程序实时展示各类关键数据，包括传感器采集的实时数据、设备运行状态、历史及当前预警记录等信息，方便用户随时查看；同时支持远程控制相关设备，如启动灭火装置、开启通风设备等操作，为保障操作安全。

四、系统设计

4.1 系统总体架构设计

系统采用四层架构设计，各层职责明确、协同工作，确保系统的稳定性与可扩展性：

4.1.1 感知层

感知层核心设备涵盖火焰传感器、MQ-2 烟雾传感器、DHT11 温湿度传感器、BH1750 光照传感器、声音传感器及 ESP32 主控制器，支持多设备分布式部署，可通过在建筑不同区域、不同角度合理布设不同设备，实现对火灾相关多维数据的全方位、无死角采集。该层级作为系统的数据采集源头，核心功能是实时捕捉火焰信号、烟雾浓度、环境温湿度、光照强度及异常声音等关键信息，通过多设备协同采集提升数据覆盖的全面性与监测的精准度，同时对采集到的原始数据进行初步筛选、预处理后上传至上层架构，还能接收平台层下发的设备校准、参数配置等控制指令，持续保障数据采集质量与整个感知设备网络的稳定运行。

4.1.2 网络层

网络层核心组件由 MQTT 代理服务器、WiFi 通信模块、UDP 协议及 MQTT 协议构成。其核心职责是搭建感知层与平台层之间的通信桥梁，实现两者间的数据高效传输。通过多种通信协议协同工作，既能借助 MQTT 协议满足物联网设备低功耗、高可靠的远程通信需求，利用 UDP 协议提升高实时性数据的传输效率，又能通过 WiFi 通信模块适配不同场景的部署需求，保障数据传输的实时性、可靠性与安全性，同时支持多设备同时接入与数据并发传输。

4.1.3 平台层

平台层作为系统的核心枢纽，核心组件涵盖 FLASK 应用、SQLite 数据库、WebSocket 实时通信模块、RESTful API 及 AI 算法引擎。该层级承担着连接感知层与应用层的关键作用，主要功能包括接收感知层上传的各类数据，通过 AI 算法引擎对数据进行智能分析以识别火灾风险，利用 SQLite 数据库实现数据的存储与管理，处理系统核心业务逻辑，同时通过 RESTful API 和 WebSocket 模块提供接口服务与实时通信支持，为上层应用提供稳定的数据与功能支撑。

4.1.4 应用层

应用层核心组件包含 Web 监控界面、微信小程序、报警系统及实时视频监控模块。该层级聚焦于用户交互与需求落地，为用户提供可视化操作界面与便捷的交互功能。用户可通过 Web 监控界面或微信小程序实现传感器数据、设备状态的实时监控，接收火灾预警信息，执行远程控制操作，查询历史数据与预警记录；同时结合报警系统实现预警联动，全面满足用户的各类使用需求。

4.2 系统模块设计

4.2.1 数据采集与预处理模块

数据采集与预处理模块承担数据获取与标准化的核心职责，数据采集环节通过 ESP32 主控制器连接各传感器，按配置频率同步获取多维传感器数据；数据预处理环节采用滑动平均法降噪、3σ原则剔除异常值，再将数据归一化映射至[0,1]区间，生成标准化数据供 AI 算法分析；同时实时监测传感器与 ESP32 主控制器在线状态，若出现设备离线或数据传输异常，立即生成故障提醒并推送至相关用户，保障数据采集的稳定性与准确性。

4.2.2 AI 智能识别与预警模块

AI 智能识别与预警模块是实现火灾精准预警的关键，系统启动时自动加载训练好的 AI 深度学习模型，且支持模型在线更新与版本管理；通过接收预处理后的实时数据，模型经多特征融合分析判断火灾风险及预警等级；满足预警条件时，自动生成含预警时间、位置、等级等信息的记录，并按配置方式推送预警信息；同时支持用户手动标记预警为误报或真报，系统将标记数据作为增量样本微调 AI 模型，持续提升模型识别准确率。

4.2.3 实时监控与远程控制模块

实时监控与远程控制模块聚焦用户交互与现场掌控需求，实时数据可视化功能可在 Web 界面与微信小程序中，通过折线图、仪表盘等形式，直观展示各传感器数据、设备状态及预警信息；远程控制功能支持用户通过上述终端，向感知层设备发送启动灭火装置、关闭阀门、设备校准等指令，且指令执行结果会实时反馈给用户，实现对现场状态的远程实时管控与快速响应。

4.3 数据库设计

4.3.1 数据库设计概述

本系统数据库采用 SQLite，围绕“数据采集-智能分析-预警处置”的核心业务流程，设计三大核心数据表及扩展数据表，实现传感器数据、报警事件、设备信息的全面存储与高效管理。数据库设计遵循规范化原则，确保数据完整性、一致性与可扩展性，同时适配 AI 低误报算法的数据需求与多角色用户的操作权限，为系统精准预警、设备监控、历史追溯提供数据支撑。

4.3.2 主要的数据表结构

（1）sensor_data(传感器数据表)：记录了实时传感器读数的信息，ID 为主键，表结构如下表 1 所示。

表 1 sensor_data 表

字段名	类型	长度	字段注释
id	INTEGER	-	主键
device_id	VARCHAR	50	关联设备 ID
device_type	VARCHAR	20	设备类型
flame_value	INTEGER	-	火焰传感器数值
smoke_value	INTEGER	-	烟雾传感器数值

temperature	REAL	-	温度值
humidity	REAL	-	湿度值
light_level	REAL	-	光照强度
alert_status	BOOLEAN	-	预警状态
timestamp	DATETIME	-	数据采集时间

(2) alert_history(报警历史表): 记录了所有报警事件的信息，ID 为主键，表结构如下表 2 所示。

表 2 alert_history 表

字段名	类型	长度	字段注释
id	INTEGER	-	主键
device_id	VARCHAR	50	关联设备 ID
alert_type	VARCHAR	20	报警类型
severity	VARCHAR	10	严重程度
flame_value	INTEGER	-	报警时火焰传感器数值
smoke_value	INTEGER	-	报警时烟雾传感器数值
temperature	REAL	-	报警时温度值
humidity	REAL	-	报警时湿度值
light_level	REAL	-	报警时光照强度
location	VARCHAR	100	报警位置
timestamp	DATETIME	-	报警发生时间
resolved	BOOLEAN	-	报警状态
resolved_time	DATETIME	-	报警处理时间

(3) device_info (设备信息表): 记录了设备注册的信息，ID 为主键，表结构如下表 3 所示。

表 3 device_info 表

字段名	类型	长度	字段注释
id	INTEGER	-	主键
device_id	VARCHAR	50	设备唯一编号
name	VARCHAR	100	设备名称
location	VARCHAR	200	安装位置
ip_address	VARCHAR	15	设备 IP 地址
device_type	VARCHAR	20	设备类型
master_id	VARCHAR	50	关联主机 ID
last_seen	DATETIME	-	设备最后在线时

			间
status	VARCHAR	20	设备状态
config	TEXT	-	设备配置参数
created_at	DATETIME	-	设备注册时间

五、使用说明

5.1 使用说明

5.1.1 主机统计

Web 主机统计如图 5-1 所示。

图 5-1 web 主机统计图

Web 主机数据可视化如图 5-2 所示。

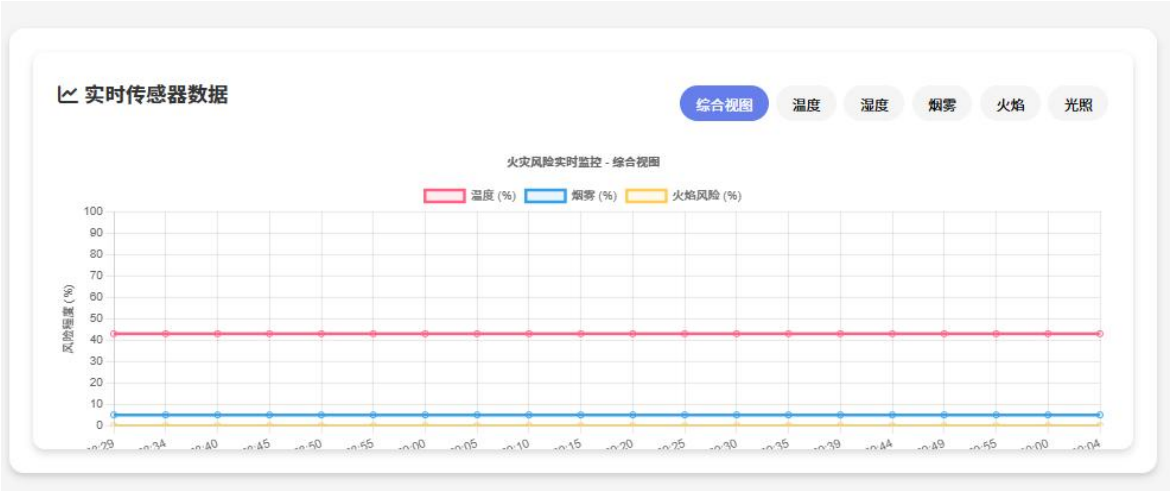

图 5-2 Web 主机数据可视化图

Web 主机数据如图 5-3 所示。

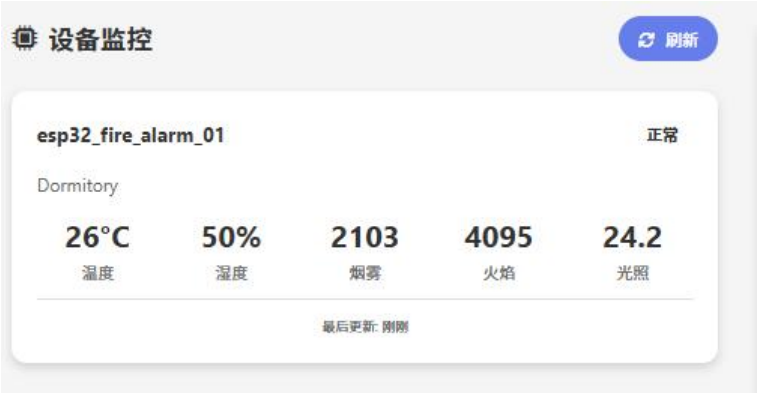

图 5-3 Web 主机数据图

Web 主机报警如图 5-4 所示。

图 5-4 Web 主机报警图

小程序主机数据的页面如图 5-5 所示。

图 5-5 小程序主机数据页面

小程序主机控制的页面如图 5-6 所示。

图 5-6 小程序主机控制页面

5.1.2 设备历史数据

Web 设备历史数据设备列表如图 5-7 所示。

图 5-7 Web 设备历史数据设备列表图

Web 设备历史数据如图 5-8 所示。

图 5-8 Web 设备历史数据图

小程序设备历史数据如图 5-9 所示。

图 5-9 小程序设备历史数据页面

5.1.3 从机数据监控

Web 从机统计如图 5-10 所示。

图 5-10 Web 从机统计图

Web 从机数据如图 5-11 所示。

图 5-11 Web 从机数据图

Web 从机报警信息如图 5-12 所示。

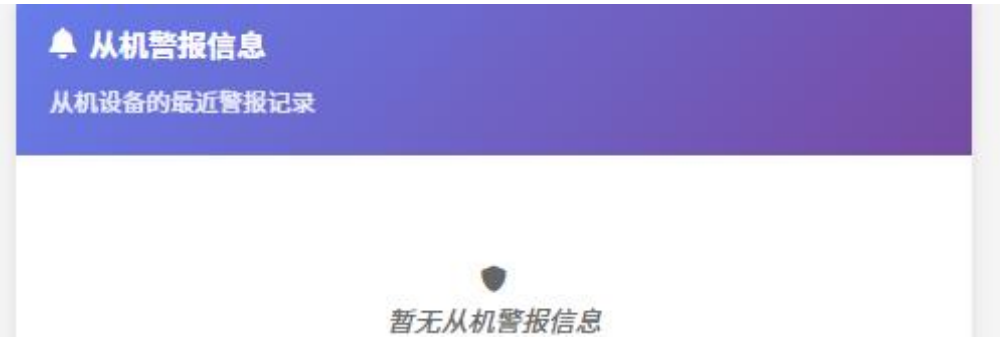

图 5-12 Web 从机报警信息图

Web 从机展示如图 5-13 所示。

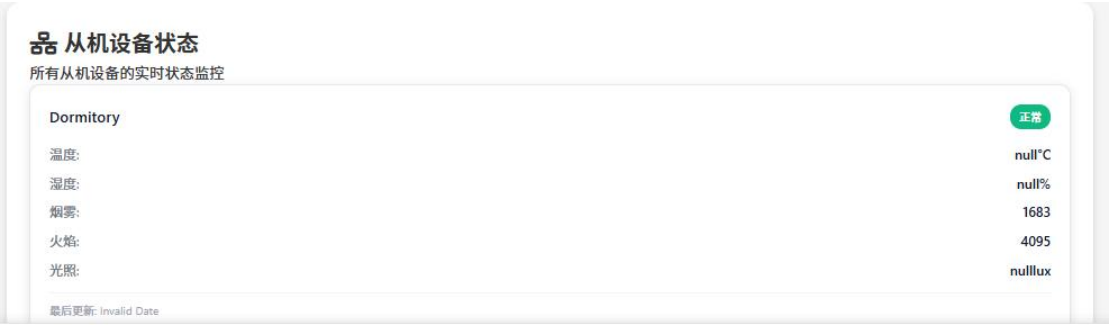

图 5-13 Web 从机展示图

小程序从机数据的页面如图 5-14 所示。

图 5-14 小程序从机数据页面

5.1.4 AI 数据分析建议页面

Web 设备列表如图 5-15 所示。

图 5-15 Web 设备列表图

Web 设备健康分析如图 5-16 所示。

图 5-16 Web 设备健康分析图

Web 环境安全分析如图 5-17 所示。

图 5-17 Web 环境安全分析图

Web 传感器统计如图 5-18 所示。

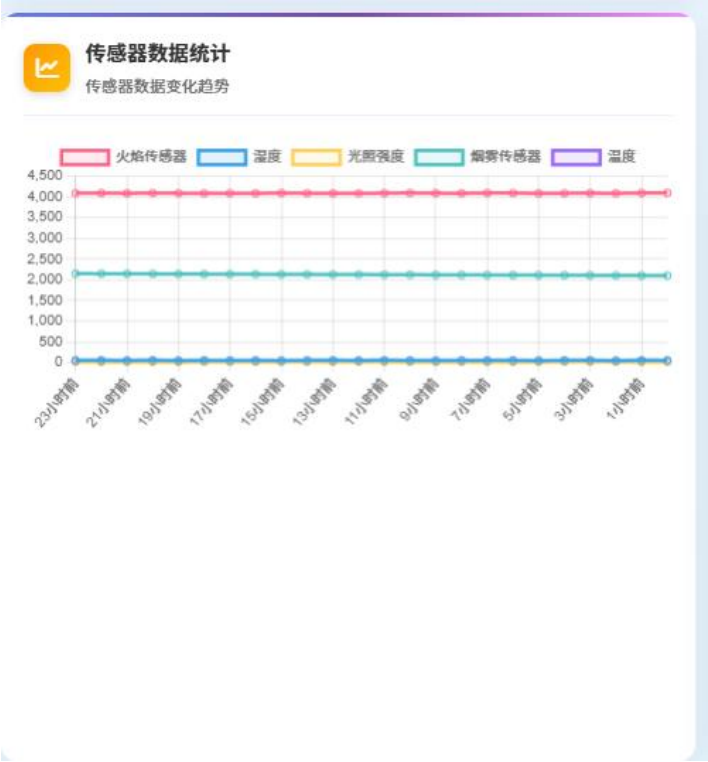

图 5-18 Web 环境安全分析图

Web AI 建议如图 5-19 所示。

图 5-19 AI 建议图

小程序分析如图 5-20 所示。

图 5-20 小程序分析图

小程序传感器统计如图 5-21 所示。

图 5-21 小程序传感器统计图

小程序 AI 建议如图 5-22 所示。

图 5-22 小程序 AI 建议图